

GitHub Copilot with C# and LINQ

Learning Objectives

By the end of this lesson, you should be able to:

- Use GitHub Copilot to assist with C# console application development.
- Guide Copilot using comments and progressively refined prompts.
- Use Copilot to generate and manipulate in-memory collections.
- Apply LINQ expressions to filter and transform data.
- Recognize when Copilot produces a technically correct solution that does not match the intended approach.
- Provide additional domain context when Copilot misunderstands the meaning of data.
- Review and test Copilot-generated C# code using dotnet run.

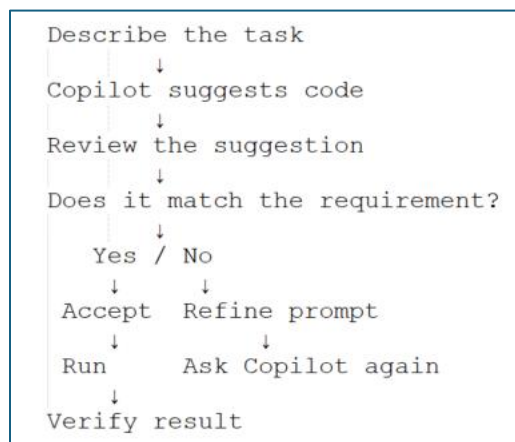
The examples use a C# console application and primarily work inside Program.cs. The emphasis is on using Copilot as a **pair-programming assistant**, while keeping the developer responsible for reviewing and validating generated code.

WARNING — Copilot Is an Assistant, Not the Decision Maker

Do not accept code simply because it compiles or appears reasonable. Check whether it implements the specific algorithm, data model, and business meaning you intended.

1. The Development Pattern

Throughout this lesson, we will use the following iterative workflow:



The important concept is **prompt refinement**. A first suggestion can be logically correct while still differing from the implementation approach we want.

2. Example 1 — Squares of Even Numbers Using LINQ

2.1 Problem

We want a **C#** program that:

1. creates numbers from 1 through 10;
2. identifies the even numbers;
3. calculates their squares;
4. displays the original even number and its square.

For example:

Number	Square
2	4
4	16
6	36
8	64
10	100

Conceptually:

```
1, 2, 3, 4, 5, 6, 7, 8, 9, 10
  ↓
Filter even numbers
  ↓
2, 4, 6, 8, 10
  ↓
Square
  ↓
4, 16, 36, 64, 100
```

3. Step 1 — Give Copilot the Initial Requirement

Open Program.cs and describe the objective using a comment:

```
// This program displays the squares of
// even numbers from 1 to 10.
```

Copilot may immediately generate a solution using a traditional for loop.

For example:

```
for (int i = 1; i <= 10; i++)
{
    if (i % 2 == 0)
    {
        Console.WriteLine(i * i);
    }
}
```

This code can produce the correct answer.

However, it does **not** satisfy our intended implementation strategy: we specifically want an in-memory collection followed by **LINQ operations**.

NOTE — Correct Output Does Not Necessarily Mean Correct Solution

Requirements may specify not only *what* a program should produce but also *how* it should be implemented. If the exercise requires LINQ, a correct for-loop solution still does not satisfy that requirement.

4. Step 2 — Refine the Prompt

Instead of accepting the first suggestion, make the requirement more specific:

```
// Create a C# list of numbers from 1 to 10.
```

A suitable starting collection could be:

```
List<int> numbers =
    new List<int> { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
```

Or, depending on the generated solution:

```
var numbers = Enumerable.Range(1, 10).ToList();
```

The important point is that we now have an in-memory collection that can be queried with LINQ.

5. Step 3 — Filter the Even Numbers

Refine the prompt again:

```
// Use LINQ to filter the even numbers.
```

A LINQ query expression may be generated:

```
var evenNumbers =  
    from number in numbers  
    where number % 2 == 0  
    select number;
```

Understanding the Query

The expression:

```
from number in numbers
```

means:

Examine each value in numbers.

The condition:

```
where number % 2 == 0
```

keeps only values whose remainder after division by 2 is zero.

Therefore:

Input

```
1 2 3 4 5 6 7 8 9 10
```

↓

```
number % 2 == 0
```

↓

```
2 4 6 8 10
```

6. Step 4 — Transform the Even Numbers

Filtering alone is not enough. We want their **squares**.

Refine the prompt:

```
// Use LINQ to filter the even numbers
```

```
// and select the square of each number.
```

Copilot may generate:

```
var squares =  
    from number in numbers  
    where number % 2 == 0  
    select number * number;
```

The query now performs two operations:

```
numbers  
↓  
WHERE  
filter even numbers  
↓  
SELECT  
calculate number × number  
↓  
squares
```

For example:

```
2 → 4  
4 → 16  
6 → 36  
8 → 64  
10 → 100
```

7. Step 5 — Display the Results

Ask Copilot to display the squares:

```
// Display the squares of the even numbers.
```

A straightforward implementation could be:

```
foreach (var square in squares)  
{  
    Console.WriteLine(square);  
}
```

which produces:

4
16
36
64
100

8. Challenge Copilot: Recover the Original Number

Suppose we want:

2 -> 4
4 -> 16
6 -> 36
8 -> 64
10 -> 100

but our LINQ expression retained only:

4, 16, 36, 64, 100

The original number is no longer explicitly stored.

In the demonstrated experiment, Copilot recognized that for these non-negative squares it could recover the original number using:

```
Math.Sqrt(square)
```

Conceptually:

```
Square = 36
```

```
Math.Sqrt(36)
```

↓

6

This is an interesting example of Copilot using the existing code context to derive information that is no longer explicitly represented.

NOTE — Generated Workarounds May Not Be the Best Design

Recovering a number using `Math.Sqrt()` works for this constrained example, but a cleaner design could preserve both the original number and its square during the LINQ projection. The important lesson here is how Copilot reasoned from the available context, not that square-root reconstruction should generally be preferred.

9. Step 6 — Run the Application

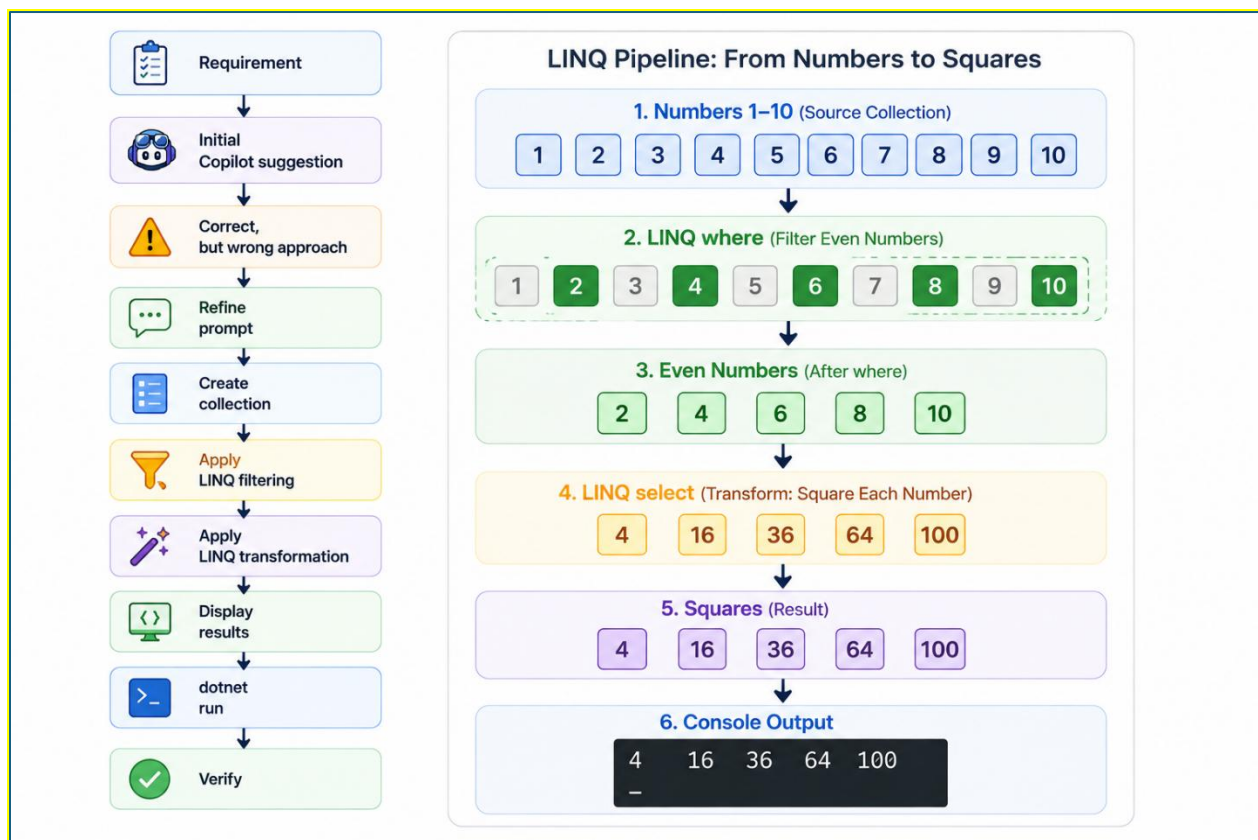
Open the integrated terminal and execute:

```
dotnet run
```

Verify the output:

```
2 4
4 16
6 36
8 64
10 100
```

The first exercise therefore follows:



10. Example 2 — Employee Data and Salary History

The second exercise introduces a more important issue: **Copilot may understand the syntax of our data while misunderstanding its meaning.**

We want to:

1. create an Employee model;
2. create five employees;
3. maintain salary history for each employee;
4. query the employee data using LINQ.

11. Step 1 — Describe the Employee Model

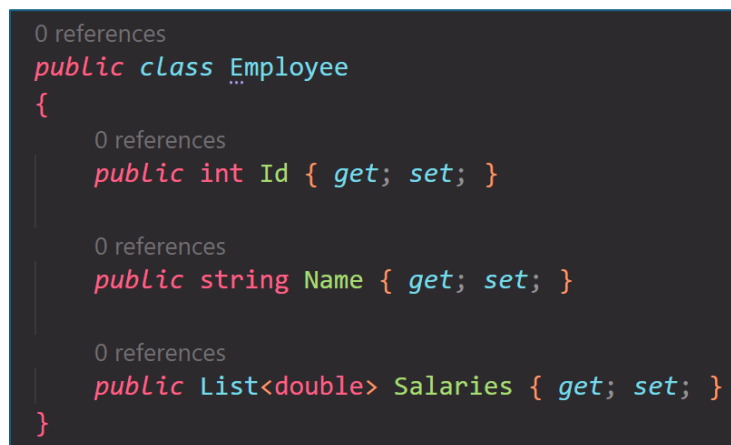
Start with a comment describing the desired class:

```
// Employee model with properties:  
// ID  
// Name  
// List of salary history as double
```

Copilot may initially misunderstand the salary requirement.

Refine the prompt until the intended structure is clear.

A suitable model is:



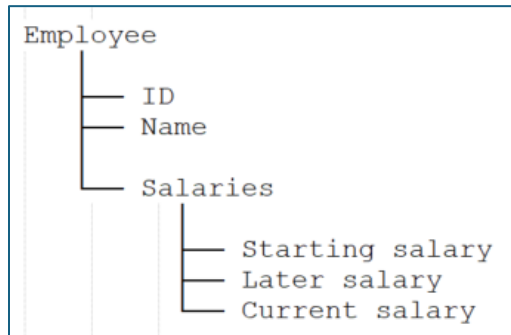
```
0 references  
public class Employee  
{  
    0 references  
    public int Id { get; set; }  
  
    0 references  
    public string Name { get; set; }  
  
    0 references  
    public List<double> Salaries { get; set; }  
}
```

The important property is:

```
public List<double> Salaries { get; set; }
```


We are not storing only one salary.

Instead:



12. Step 2 — Generate Sample Employees

Ask Copilot to create **five employees**.

For example, the resulting data may conceptually resemble:

```
var employees = new List<Employee>
{
    new Employee
    {
        Id = 1,
        Name = "John",
        Salaries = new List<double>
        {
            1000, 2000, 3000
        }
    },

    new Employee
    {
        Id = 2,
        Name = "Jack",
        Salaries = new List<double>
        {
            2000, 4000, 5000
        }
    },

    // Additional employees...
};
```

The exact names and salary values may differ because Copilot generates the sample data.

Verify that:

- there are five employees;
- IDs are appropriate;
- each employee has a name;
- each employee has multiple salary values.

13. Understanding Salary History

The meaning of the salary collection is critical.

Suppose:

```
Salaries = new List<double>
{
    3000,
    4000,
    5000
};
```

Our intended interpretation is:



Therefore:

```
employee.Salaries.First()
```

represents the **first/oldest salary**, while:

```
employee.Salaries.Last()
```

represents the **current/newest salary**.

This semantic information becomes important when prompting Copilot.

14. Step 3 — Retrieve Employee Name and First Salary

Give Copilot the requirement:

```
// Get employee name and first salary.
```

A suitable LINQ expression is:

```
var result = employees.Select(employee => new
{
    employee.Name,
    FirstSalary = employee.Salaries.First()
});
```

Why Use Select()?

Select() performs a projection.

We begin with an Employee:

```
Employee
├─ Id
├─ Name
└─ Salaries
```

and transform it into a smaller result:

```
Result
├─ Name
└─ FirstSalary
```

The expression:

```
employee.Salaries.First()
```

retrieves the first salary in the history.

15. Step 4 — Display the Query Result

Use a foreach loop:

```
foreach (var employee in result)
{
    Console.WriteLine(
        $"{employee.Name}: {employee.FirstSalary}"
    );
}
```

Then execute:

```
dotnet run
```

Compare the console output with the employee data.

This is essential: Copilot may have generated syntactically valid LINQ, but you still need to confirm that it selected the intended values.

16. Step 5 — Query Employees with Current Salary $\geq$ 5000

Now change the requirement.

We want:

Employees whose **current salary** is 5000 or above.

A first prompt might be:

// Get employee name and salary of 5000 or above.

This prompt contains an important ambiguity.

What does:

salary of 5000 or above

mean when each employee has several salary values?

Copilot may interpret it as:

Find any salary in the employee's salary list that is at least 5000.

That is not necessarily our requirement.

17. The Semantic Context Problem

Consider:

Jill's Salary History

4000 → 5000 → 6000

We want to evaluate:

Current salary = 6000

not simply search the collection for every value satisfying:

salary $\geq$ 5000

The difference is:

Wrong interpretation:

```
"Find salary values >= 5000"
```

versus

Required interpretation:

```
"Look at each employee's CURRENT salary,  
then determine whether it is >= 5000."
```

This is not primarily a C# syntax problem.

It is a **domain-context problem**.

Copilot sees:

```
List<double> Salaries
```

but the type itself does not explain that:

```
First element = oldest salary  
Last element  = current salary
```

18. Step 6 — Add the Missing Context

Provide Copilot with the missing semantic information:

```
// The salary list is ordered from  
// oldest salary to newest salary.
```

Then refine the query:

```
// Get employee name and current salary  
// for employees whose current salary  
// is 5000 or above.
```

Now Copilot has enough context to understand that the current salary should be obtained from the **last element**.

A suitable LINQ solution is conceptually:

```
var result = employees
    .Where(employee =>
        employee.Salaries.Last() >= 5000)
    .Select(employee => new
    {
        employee.Name,
        CurrentSalary =
            employee.Salaries.Last()
    });
```

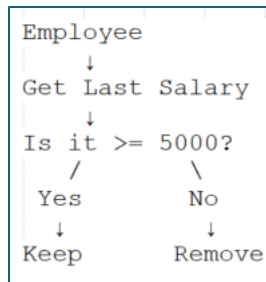
19. Understanding the LINQ Pipeline

The query has two major stages.

Stage 1 — Filter

```
.Where(employee =>
    employee.Salaries.Last() >= 5000)
```

For each employee:



Stage 2 — Project

```
.Select(employee => new
{
    employee.Name,
    CurrentSalary = employee.Salaries.Last()
});
```

For employees that passed the filter, retain only:

Name + Current Salary

20. Step 7 — Run and Verify

Display the resulting employees:

```
foreach (var employee in result)
{
    Console.WriteLine(
        $"{employee.Name}: {employee.CurrentSalary}"
    );
}
```

Then execute:

dotnet run

In the demonstrated example, the correct result contains the three employees **Jack, Jill, and Joe**, because their current salary is at least 5000.

Do not verify only the number of employees. Compare each returned employee against the original salary history.

21. Why Additional Context Changed the Result

This is the most important lesson in the second exercise.

Initially, Copilot knew:

Employee

↓

List<double> Salaries

After our clarification, it knew:

Employee

↓

Salary History

↓

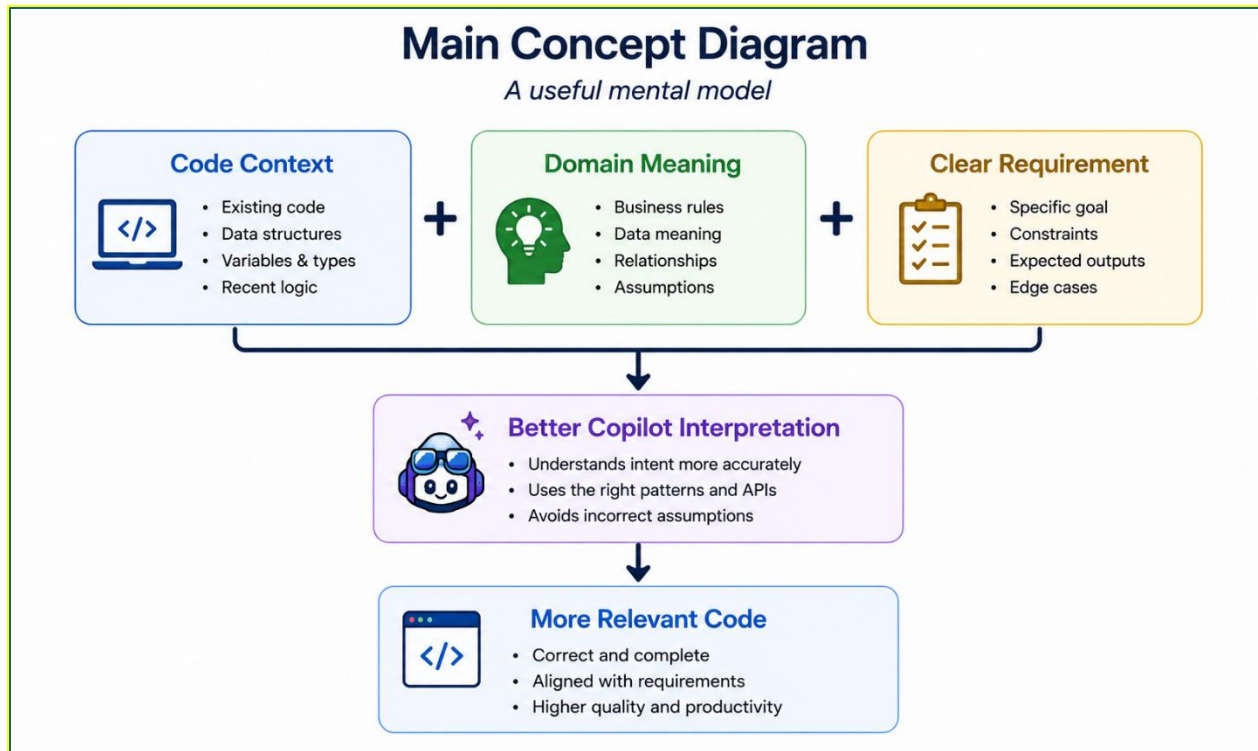
Ordered oldest → newest

↓

Last() = current salary

The prompt became more semantically precise, allowing Copilot to generate a query that better matched the actual requirement.

A mental model can be:



22. Common Pitfalls

PITFALL — Accepting the First Working Solution

Copilot's initial for-loop solution correctly calculated the squares, but the exercise required a collection and LINQ. Always compare generated code with the complete requirement.

PITFALL — Assuming Copilot Understands Your Data Model

A property called `Salaries` does not automatically communicate that the list represents chronological salary history.

TIP — Explain Relationships and Ordering

If the meaning of a collection depends on order, state that explicitly. For example: *"The salary list is ordered from oldest salary to newest salary."*

WARNING — Always Execute and Verify

A LINQ expression can compile successfully while implementing the wrong business rule. Run the application and compare the results with the underlying data.

23. Practical Exercise

Starting from a C# console application:

1. Create a list containing numbers 1 through 10.
2. Ask Copilot to use LINQ to filter the even numbers.
3. Refine the prompt so the query calculates their squares.
4. Display the results.
5. Run the application with: `dotnet run`

Then create an Employee model containing:

- ID
- Name
- Salary History

Generate five sample employees and use Copilot to create LINQ queries that:

- retrieve each employee's name and first salary;
- retrieve employees whose **current salary** is at least 5000.

If Copilot misinterprets "current salary," add the missing context about the chronological ordering of the salary list and regenerate the query.

24. Key Takeaways

This lesson demonstrates that effective Copilot usage involves more than asking it to "write the code."

The productive workflow is:

